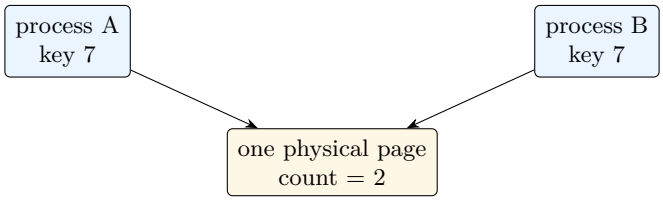
A1-10 shared page – the bare minimum

Only what you need to make it work. Every line explained: `walkthrough.pdf` in this folder. Start: `task start a1-10`. Done: `task test` shows **9 of 9**.

The idea in 3 sentences

1. `shm_attach(7)` in two processes puts the **same physical page** into both – what one writes, the other reads.
2. The kernel keeps **one small table**: for each key (1–15) the physical page and **how many processes** use it.
3. The page is only freed when that count drops to **0** – so unmapping it must **not** free it.



Checklist – 6 places

#	What	File
1	numbers, libc, cases (plumbing)	`syscall-definitions.h`, `nonstd.h/.c`, `Syscall.cpp`
2	unmap *without* freeing	`ArchMemory.h/.cpp`
3	the table (new class)	new `common/include/mm/SharedPages.h` + `common/source/mm/SharedPages.cpp`
4	create the table at boot	`common/source/kernel/main.cpp`
5	attach + detach	`UserProcess.h/.cpp`
6	clean up when a process ends	`UserProcess.cpp` (destructor)

1 – Plumbing `syscall-definitions.h · nonstd.h · nonstd.c · Syscall.cpp`

```
// syscall-definitions.h
#define sc_shm_attach 1680
#define sc_shm_detach 1681

// nonstd.h  (needs #include "types.h" at the top)
extern void* shm_attach(size_t key);
extern int shm_detach(size_t key);

// nonstd.c
void* shm_attach(size_t key) { return (void*) __syscall(sc_shm_attach, key, 0, 0, 0,
    ↪ 0); }
int shm_detach(size_t key)   { return __syscall(sc_shm_detach, key, 0, 0, 0, 0); }

// Syscall.cpp, in the switch
case sc_shm_attach: return_value = currentThread->getProcess()->shmAttach(arg1); break;
case sc_shm_detach: return_value = currentThread->getProcess()->shmDetach(arg1); break;
```

2 – Unmap without freeing

ArchMemory.h · ArchMemory.cpp

```
// ArchMemory.h - add a parameter with a default (old calls keep working)
bool unmapPage(uint64 virtual_page, bool free_physical_page = true);

// ArchMemory.cpp - first line of unmapPage, and wrap the freePPN line in an if
bool ArchMemory::unmapPage(uint64 virtual_page, bool free_physical_page)
  ...
  if (free_physical_page)
    PageManager::instance()->freePPN(m.page_ppn);
```

Why: a shared page still belongs to the other process.

3 – The table

new files SharedPages.h / SharedPages.cpp in mm/

```
// common/include/mm/SharedPages.h
#pragma once
#include "types.h"
class Mutex;
#define SHM_MAX_KEYS 16
#define SHM_BASE 0x0000400000000000ULL      // key k lives at SHM_BASE + k pages

class SharedPages
{
  public:
    static SharedPages* instance();
    static void init();
    size_t acquire(size_t key);              // count + 1 (creates the page), returns
    ↪ the ppn
    void release(size_t key);                // count - 1 (frees the page at 0)
  private:
    SharedPages();
    static SharedPages* instance_;
    struct Entry { size_t ppn; size_t references; };
    Mutex* lock_;
    Entry entries_[SHM_MAX_KEYS];
};
```

```
// common/source/mm/SharedPages.cpp
#include "SharedPages.h"
#include "PageManager.h"
#include "Mutex.h"
#include "ScopeLock.h"
#include "assert.h"

SharedPages* SharedPages::instance_ = nullptr;
void SharedPages::init() { instance_ = new SharedPages(); }
SharedPages* SharedPages::instance() { return instance_; }

SharedPages::SharedPages() : lock_(new Mutex("SharedPages::lock_"))
{
  for (Entry& entry : entries_) { entry.ppn = 0; entry.references = 0; }
}

size_t SharedPages::acquire(size_t key)
{
  ScopeLock lock(*lock_);
  if (entries_[key].ppn == 0)
    entries_[key].ppn = PageManager::instance()->allocPPN();   // new, zeroed page
  entries_[key].references++;
```

```
  return entries_[key].ppn;
}

void SharedPages::release(size_t key)
{
  ScopeLock lock(*lock_);
  entries_[key].references--;
  if (entries_[key].references == 0)
  {
    PageManager::instance()->freePPN(entries_[key].ppn);       // last user gone
    entries_[key].ppn = 0;
  }
}
```

4 – Create the table at boot

main.cpp

```
#include "SharedPages.h"          // at the top
...
Scheduler::instance();
SharedPages::init();               // new line, in startup()
```

Why: SWEB does not run constructors of global objects.

5 – Attach and detach

UserProcess.h · UserProcess.cpp

```
// UserProcess.h - public:
size_t shmAttach(size_t key);
size_t shmDetach(size_t key);
// UserProcess.h - private, at the bottom:
bool shm_attached_[16] = {};               // which keys this process has mapped
```

```
// UserProcess.cpp - #include "SharedPages.h" and "ScopeLock.h" at the top
size_t UserProcess::shmAttach(size_t key)
{
  if (key == 0 || key >= SHM_MAX_KEYS)
    return 0;
  size_t vpn = SHM_BASE / PAGE_SIZE + key;
  size_t ppn = SharedPages::instance()->acquire(key);     // 1. count + 1
  bool already;
  {
    ScopeLock lock(thread_lock_);                         // 2. then map (other lock)
    already = shm_attached_[key];
    if (!already)
    {
      bool mapped = arch_memory_->mapPage(vpn, ppn, 1);
      assert(mapped);
      shm_attached_[key] = true;
    }
  }
  if (already)
    SharedPages::instance()->release(key);                // counted twice: undo
  return vpn * PAGE_SIZE;
}

size_t UserProcess::shmDetach(size_t key)
{
  if (key == 0 || key >= SHM_MAX_KEYS)
    return (size_t) -1;
  {
```

```
    ScopeLock lock(thread_lock_);                          // 1. unmap, keep the page
    if (!shm_attached_[key])
      return (size_t) -1;
    arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
    ArchMemory::flushTlb();
    shm_attached_[key] = false;
  }
  SharedPages::instance()->release(key);                   // 2. count - 1
  return 0;
}
```

6 – Clean up when a process ends

`UserProcess.cpp, ~UserProcess`

Between `delete loader_;` and `delete arch_memory_;`:

```
for (size_t key = 1; key < SHM_MAX_KEYS; ++key)
{
  if (shm_attached_[key])
  {
    arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
    SharedPages::instance()->release(key);
  }
}
```

Why: `delete arch_memory_` frees every mapped page – also the shared one, if a process ends without detaching.

The three rules to remember

1. Count the users; free only at 0.
2. Unmapping a shared page never frees it (`false`).
3. Never hold both locks: attach = count, then map; detach = unmap, then count.